KHULNA
UNIVERSITY OF
ENGINEERING AND
TECHNOLOGY

Akash Biswas
Lecturer, EEE
KUET

## *Department of Electrical and Electronic Engineering*

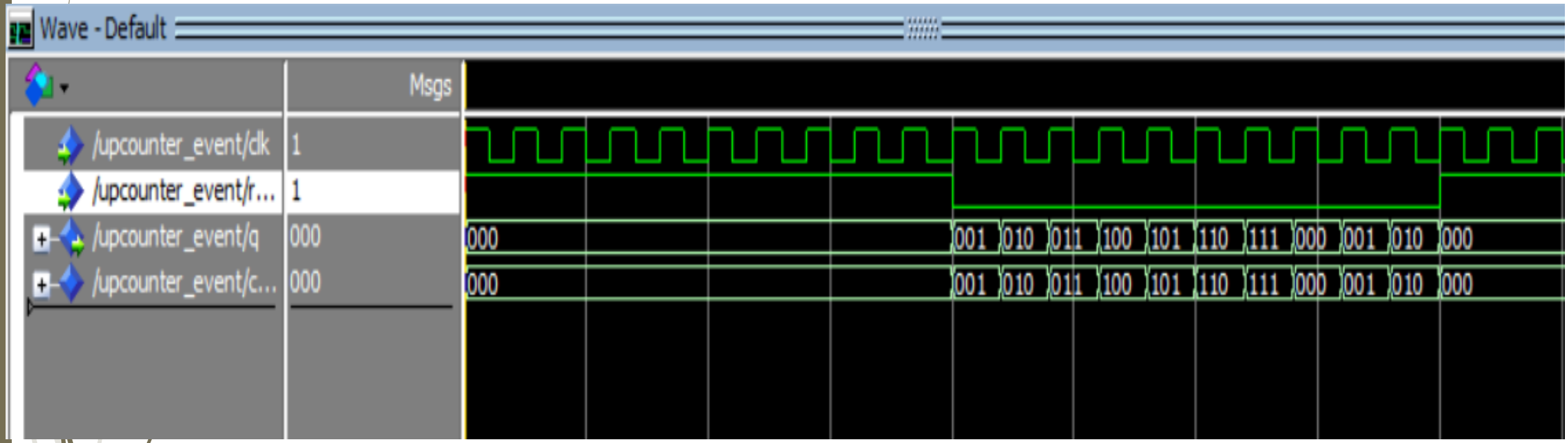
**Course Code : EE 4121**

# VLSI

**Hardware Description Language**

# Upcounter-3 Bit:

```
1   library ieee;
2   use ieee.std_logic_1164.all;
3   use ieee.std_logic_unsigned.all;  -- for easy + on std_logic_vector
4
5   entity upcounter_event is
6       port(
7           clk   : in  std_logic;
8           reset : in  std_logic;
9           q     : out std_logic_vector(2 downto 0)
10      );
11  end entity upcounter_event;
12  architecture behavioral of upcounter_event is
13      signal count : std_logic_vector(2 downto 0); -- := "000";
14  begin
15      process(clk)
16      begin
17          if clk'event and clk = '1' then        -- check rising edge of clock
18              if reset = '1' then
19                  count <= "000";                -- reset counter
20              elsif count = "111" then
21                  count <= "000";                -- roll over at max
22              else
23                  count <= count + 1;            -- increment
24              end if;
25          end if;
26      end process;
27
28      q <= count;
29  end architecture behavioral;
```


Wave - Default
Msgs
/upcounter_event/clk
1
/upcounter_event/r...
1
/upcounter_event/q
000
/upcounter_event/c...
000
000
001
010
011
100
101
110
111
000
001
010
000
000
001
010
011
100
101
110
111
000
001
010
000

## What is Verilog?

Verilog, standardized as IEEE 1364, is a hardware description language (HDL) used to model electronic systems. It is most commonly used in the design and verification of digital circuits, with the highest level of abstraction being at the register-transfer level.

In 2009, the Verilog standard (IEEE 1364-2005) was merged into the SystemVerilog standard, creating IEEE Standard 1800-2009. Since then, Verilog has been officially part of the SystemVerilog language. The current version is IEEE standard 1800-2023.

The designers of Verilog wanted a language with syntax similar to the C programming language, which was already widely used in engineering software development. Like C, Verilog is case-sensitive and has a basic preprocessor (though less sophisticated than that of ANSI C/C++). Its control flow keywords (if/else, for, while, case, etc.) are equivalent, and its operator precedence is compatible with C.

| *Operator Type* | *Symbols* | *Application / Purpose* |
|---|---|---|
| **Arithmetic** | +, -, *, /, % | Addition, subtraction, multiplication, division, modulus |
| **Relational** | >, < | Compare values (greater/less than checks) |
| **Equality** | ==, != | Compare values (ignores x and z) |
| **Logical** | &&, ` | |
| **Bitwise** | &, ` | , ^, ~, ~^or^~` |
| **Assignment** | =, <= | Blocking (=) and Non-blocking (<=) assignments |

```
Assign   sum  = a + b;        // Arithmetic
assign   flag = (a >= b);     // Relational
assign   out  = enable && clk; // Logical
assign    y   = a & b;        // Bitwise
```

| Operator | Name | Example |
|---|---|---|
| & | Bitwise AND | a & b |
| `\| | Bitwise OR ` | OR operation on each bit |
| ^ | Bitwise XOR | a ^ b |
| ~ | Bitwise NOT | ~a |
| ~^ or ^~ | Bitwise XNOR | a ~^ b |

## Verilog Design Styles

- Dataflow Modeling
- Switch Modeling
- Behavioral Modeling
- Structural Modeling
- Mixed Modeling

```
module MODULE_name (port_list);              // Module Declaration

// 1. Port Declarations
input   a, b;        // Inputs
output  c;           // Outputs

// 2. Internal Signal Declarations
wire    x;           // Net type
reg     y;           // Register type


// 3. Code Section
// - Continuous assignments (dataflow)
// - Gate-level modeling
// - Behavioral blocks (always, initial)

assign x = a & b;   // Example continuous assignment

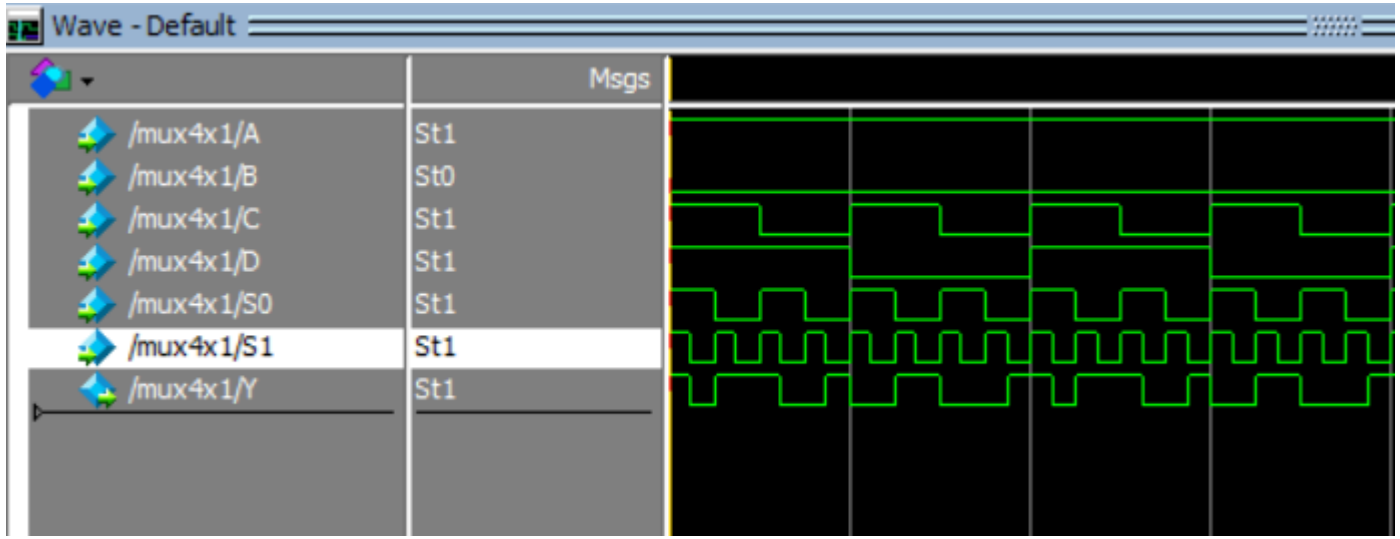
always @ (a or b) begin
    y = a | b;      // Example procedural assignment
end

assign c = y & x;    // Output logic

endmodule   // End of module
```

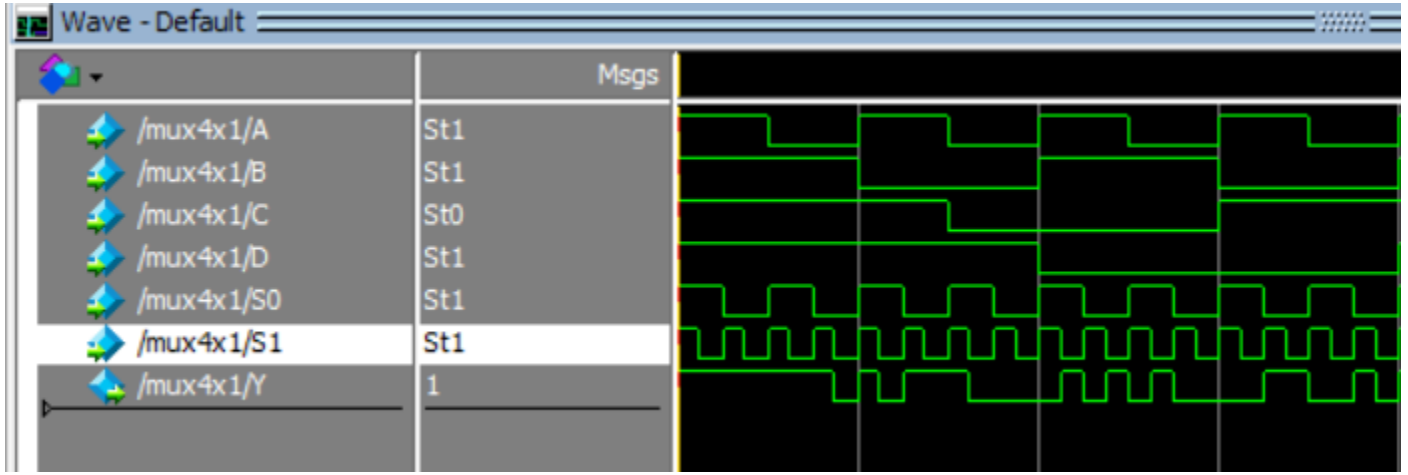
# Dataflow Modeling

```
1   module mux4x1 (
2       input  wire A, B, C, D,    // Data inputs
3       input  wire S0, S1,        // Select inputs
4       output wire Y              // Output
5   );
6
7   // Dataflow modeling using boolean expressions
8   assign Y = (~S1 & ~S0 & A) | (~S1 &  S0 & B) | ( S1 & ~S0 & C) | ( S1 &  S0 & D);
9
10  endmodule
```



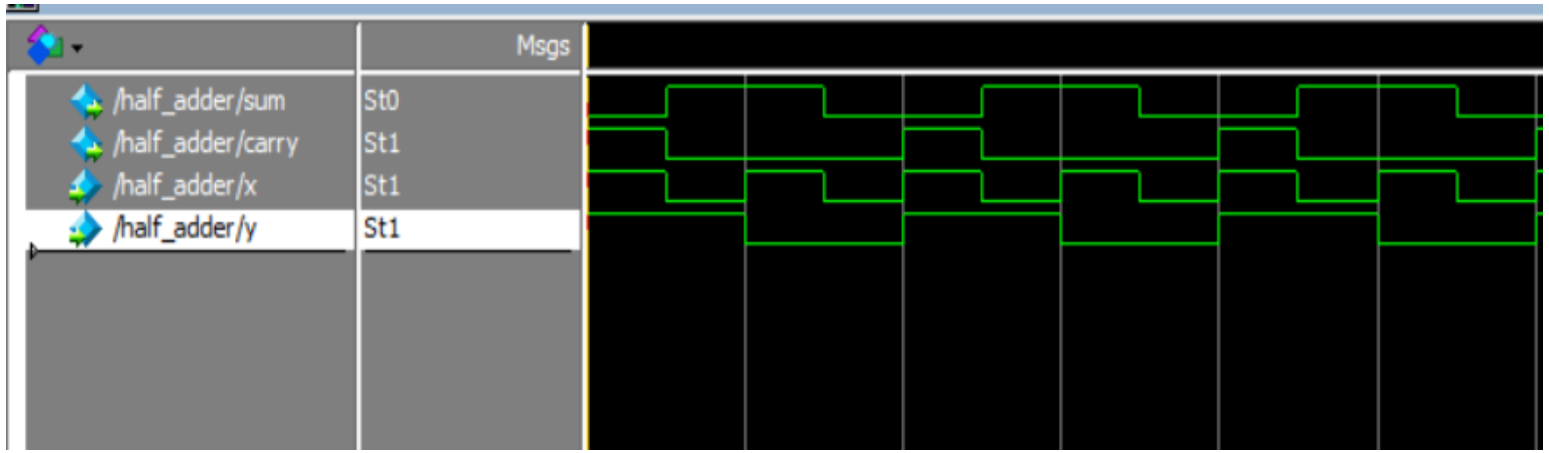
# Behavioral Modeling

```
1   module mux4x1 (
2       input  wire A, B, C, D,    // Data inputs
3       input  wire S0, S1,        // Select inputs
4       output reg  Y              // Output must be reg in always block
5   );
6
7   always @ (A or B or C or D or S0 or S1) begin
8       if (S1 == 0 && S0 == 0)
9           Y = A;          // Select A
10      else if (S1 == 0 && S0 == 1)
11          Y = B;          // Select B
12      else if (S1 == 1 && S0 == 0)
13          Y = C;          // Select C
14      else
15          Y = D;          // Select D
16  end
17
18  endmodule
```



# Structural Modeling

```
1   module xor_gate (output y, input a, b);
2     assign y = a ^ b;  // XOR gate
3   endmodule
4
5
6   module and_gate (output y, input a, b);
7     assign y = a & b; // AND gate
8   endmodule
9
10  // Half Adder
11  module half_adder (output sum, carry, input x, y);
12
13    wire w1, w2;
14
15    xor_gate u1 (w1, x, y);
16    and_gate u2 (w2, x, y);
17
18    assign sum = w1;
19    assign carry = w2;
20
21  endmodule
```




```
// Half Adder
module half_adder (output sum, carry, input x, y);

 xor_gate u1 (sum,   x, y);
 and_gate u2 (carry, x, y);
```

positional mapping

```
xor_gate u1 (.a(x), .y(sum), .b(y));
and_gate u2 (carry, x, y);
```

named mapping

# Programmable Logic Array

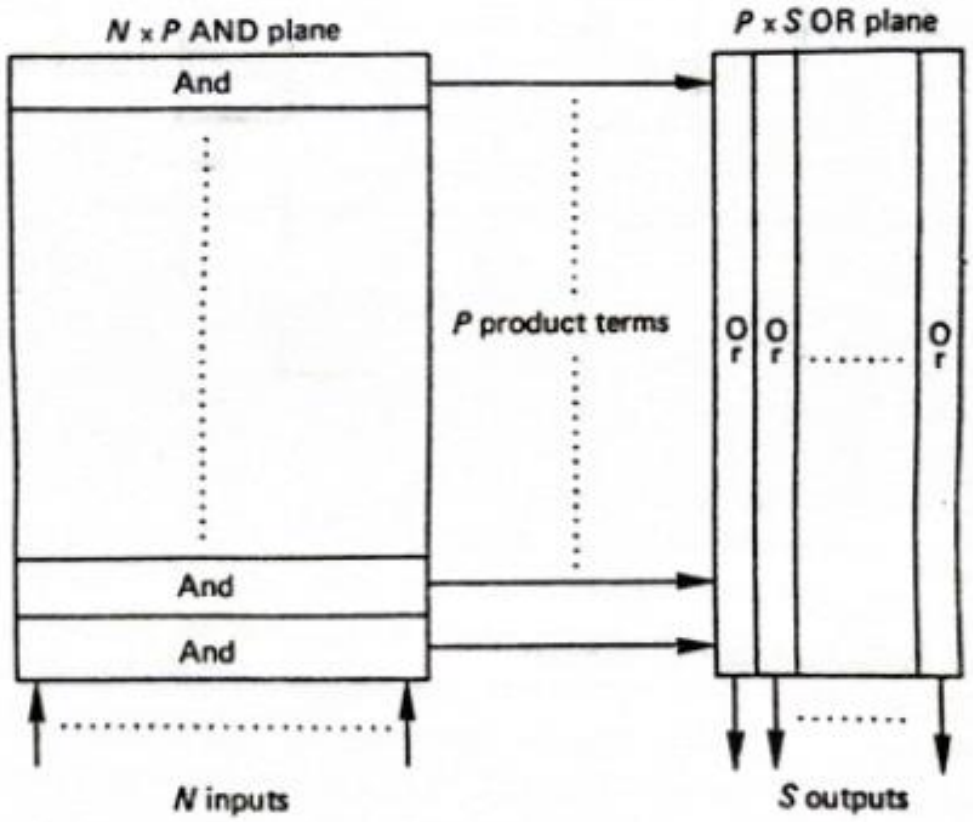



*Figure 4.5* General structure of programmable logic array

Table 4.1 Truth table for three-line priority encoder

| *Inputs* | | | | *Outputs* | |
|---|---|---|---|---|---|
| *Enable* | *D0* | *D1* | *D2* | *A* | *B* |
| 0 | X | X | X | 0 | 0 |
| 1 | 0 | 0 | 0 | 0 | 0 |
| 1 | 1 | X | X | 1 | 1 |
| 1 | 0 | 1 | X | 1 | 0 |
| 1 | 0 | 0 | 1 | 0 | 1 |

X = 0 or 1 – that is, don't care.

Inspection of the truth table reveals that

$$A = \text{Enable}.D0 + \text{Enable}.D1$$

$$B = \text{Enable}.D0 + \text{Enable}.\overline{D1}.D2$$

Thus the AND plane forms three product terms $P0$, $P1$ and $P2$ where

$$P0 = \text{Enable}.D0 = \overline{\overline{\text{Enable}} + \overline{D0}}$$

$$P1 = \text{Enable}.D1 = \overline{\overline{\text{Enable}} + \overline{D1}}$$

$$P2 = \text{Enable}.\overline{D1}.D2 = \overline{\overline{\text{Enable}} + D1 + \overline{D2}}$$

$P0$, $P1$ and $P2$ are combined in the OR plane to form the sum of products terms $S0$ and $S1$ where

$$S0 = \overline{P0 + P1} = \overline{\text{Enable}.D0 + \text{Enable}.D1}$$

$$S1 = \overline{P0 + P2} = \overline{\text{Enable}.D0 + \text{Enable}.\overline{D1}.D2}$$

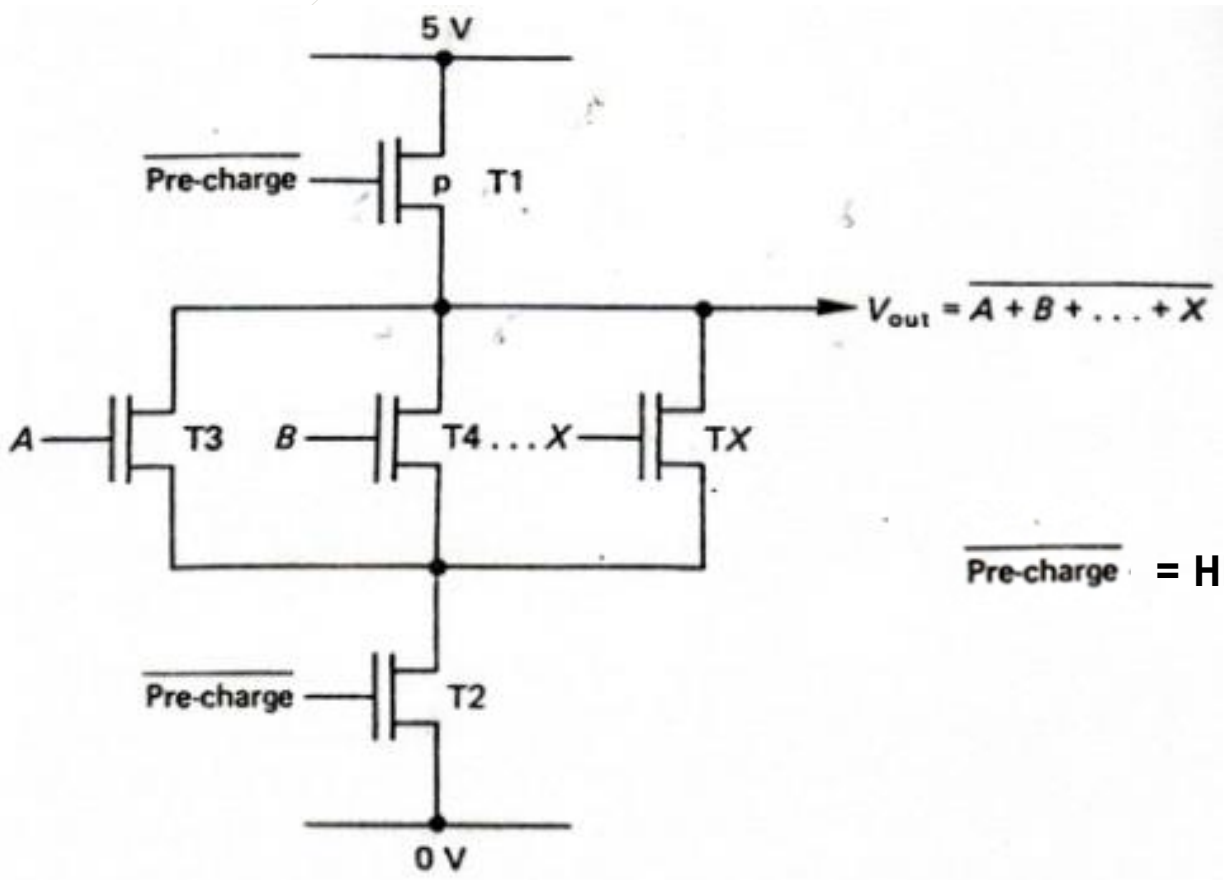



*Figure 4.8* A pre-charged nor gate

$\overline{\text{Pre-charge}}$ = Low------ T1-> ON T2-> OFF
Output = 5 V

$\overline{\text{Pre-charge}}$ = HIGH------ T1-> OFF, T2-> ON, T3...TX any one is ON
Output = 0 V

$\overline{\text{Pre-charge}}$ = HIGH------ T1-> OFF, T2-> ON
Output = 5 V